IOT-Based Smart Greenhouse Monitoring System

MD. Rahat Hossen*, Md. Asraful Islam†, Md. Mahabub Islam‡
*Dept. of IoT and Robotics Engineering
University of Frontier Technology, Bangladesh
Kaliakair, Gazipur, Dhaka-1750
Email: rahat0001@std.bdu.ac.bd, asraful0001@std.bdu.ac.bd, mahabubl0001@std.bdu.ac.bd

***Abstract*—Agriculture increasingly requires automation to ensure efficient resource utilization and consistent crop production. Traditional greenhouse management relies on manual monitoring of environmental conditions such as soil moisture, temperature, and humidity, which can lead to overwatering, energy waste, and reduced plant growth. To address these limitations, this paper presents a low-cost IoT-enabled Smart Greenhouse Monitoring System using the ESP32 microcontroller, a soil moisture sensor, a DHT22 temperature-humidity sensor, and the Blynk IoT platform. The system continuously monitors environmental parameters and automatically controls irrigation based on real-time soil conditions. It supports two modes: (i) an automatic mode where irrigation is triggered by sensor thresholds, and (ii) a manual mode allowing remote user control via the Blynk app. Experimental testing demonstrated improved water efficiency, stable environmental control, and reduced human intervention. The proposed solution is affordable, scalable, and suitable for smart agriculture applications in developing regions.**

***Index Terms*—IoT, ESP32, FreeRTOS, Smart Agriculture, Irrigation, Automation, Blynk**

I. Introduction

Agriculture plays a vital role in ensuring food security and sustainable economic development. However, traditional farming practices often face challenges such as inefficient water usage, unpredictable environmental conditions, and lack of real-time monitoring. Greenhouse technology has emerged as an effective solution to overcome these challenges by providing a controlled environment for plant growth. By regulating factors such as temperature, humidity, soil moisture, and light intensity, greenhouses enable farmers to optimize crop production and improve agricultural efficiency [13].

With the rapid development of the Internet of Things (IoT), smart agriculture systems have become increasingly popular. IoT-based greenhouse monitoring systems integrate sensors, embedded controllers, and communication technologies to continuously monitor environmental conditions and provide real-time data to farmers [1]. These systems help automate irrigation, regulate environmental parameters, and enhance decision-making processes, thereby improving crop yield and resource management [2]. Wireless sensor networks and cloud-based platforms further enable remote monitoring and control, making modern greenhouse systems more efficient and scalable [9].

Recent research has focused on integrating embedded systems and real-time operating systems to improve the reliability and responsiveness of smart agriculture solutions. Real-time operating systems such as FreeRTOS provide task scheduling, resource management, and efficient multitasking capabilities for embedded devices used in agricultural monitoring systems [3]. The use of FreeRTOS enables multiple sensing and control tasks to run simultaneously while maintaining real-time performance, which is essential for monitoring dynamic environmental conditions inside greenhouses [14].

Several IoT-based greenhouse monitoring solutions have been proposed in the literature. These systems typically utilize microcontrollers such as ESP32, environmental sensors, and cloud platforms for data visualization and control [4]. Recent studies have also explored advanced approaches including intelligent plant monitoring, cloud integration, and AI-based irrigation management to enhance sustainability and energy efficiency [6], [12]. Furthermore, IoT platforms such as Blynk and ThingSpeak allow users to remotely monitor and analyze greenhouse conditions through mobile or web interfaces [8].

Despite these advancements, many existing systems still face limitations related to scalability, real-time responsiveness, and integration of multiple environmental parameters. Therefore, there is a need for a reliable and efficient monitoring system that can simultaneously handle sensing, processing, and communication tasks in real time.

This paper presents the design and implementation of a **FreeRTOS-based IoT Smart Greenhouse Monitoring System**. The proposed system utilizes environmental sensors connected to an ESP32 microcontroller running FreeRTOS to collect and process real-time data such as temperature, humidity, and soil moisture. The system enables remote monitoring through an IoT platform, allowing users to observe environmental conditions and take appropriate actions for optimal plant growth. The implementation demonstrates how real-time task scheduling and IoT connectivity can enhance greenhouse automation and monitoring efficiency.

II. Contributions

The main contributions of this paper are as follows. First, we develop a real-time greenhouse monitoring system based on the ESP32 microcontroller running FreeRTOS, enabling deterministic multitasking for reliable sensor data acquisition and control. Second, the system integrates threshold-based irrigation logic with a cloud-connected interface (Blynk) to provide both autonomous and manual control modes. Third, experimental results demonstrate improved water use efficiency (approximately 25% savings) and rapid response to soil dryness. Finally, the design is low-cost (around $30 total) and

scalable, making smart greenhouse automation accessible for small-scale farmers in developing regions.

III. Literature Review

The application of IoT in agriculture has significantly improved environmental monitoring and irrigation automation. Zhang et al. [1] proposed an IoT-based smart agriculture monitoring system that integrates environmental sensors with a cloud platform for real-time data visualization. Their work demonstrated improved crop management through data-driven irrigation control. However, that system primarily relied on traditional sequential processing, which may limit performance under multitasking conditions. Kumar and Singh [2] developed an automated greenhouse control system using embedded technology and threshold-based irrigation logic. Their system reduced water wastage and improved environmental stability. However, it lacked advanced task scheduling mechanisms such as priority-based real-time execution. Ahmed et al. [3] introduced a FreeRTOS-based environmental monitoring system that demonstrated improved multitasking, deterministic scheduling, and faster response time compared to loop-based designs. Their findings highlight the importance of RTOS in real-time agricultural automation.

Other IoT irrigation systems using cloud services such as Blynk and ThingSpeak have shown the benefits of remote monitoring and mobile control [8]. For instance, Saputri et al. [8] designed an IoT-enabled irrigation system using Blynk and ThingSpeak for real-time sensor data and control, achieving cost-effective operation and water savings. However, many existing solutions either remain expensive or do not combine cloud connectivity with structured real-time task management. The proposed RTOS-Based Smart Greenhouse Monitoring System addresses these limitations by integrating IoT cloud communication with FreeRTOS-based multitasking, ensuring deterministic performance, reduced latency, and improved water efficiency.

TABLE I
Comparison of Related Works

Authors (Year)	System Type	Technology Used	Key Contribution	Limitations
Zhang et al. [1] (2021)	IoT Smart Agriculture	Sensors + Cloud Platform	Real-time monitoring and data-driven irrigation	Sequential execution, limited real-time control
Kumar & Singh [2] (2022)	Automated Greenhouse	Embedded system + Threshold logic	Reduced water waste through automated irrigation	No RTOS or deterministic scheduling
Ahmed et al. [3] (2023)	RTOS-Based Monitoring	FreeRTOS + Embedded platform	Improved multitasking and deterministic execution	Limited cloud-based remote control features
Saputri et al. [8] (2025)	IoT Irrigation System	ESP32 + Blynk + ThingSpeak	Remote monitoring with mobile control (water savings)	Focused on monitoring, minimal real-time control
Banluesapy et al. [12] (2025)	AI-Augmented Irrigation	ESP32 + Sensors + MQTT + ML	50% reduction in water and energy usage	Complex ML model required
Proposed Work	**RTOS-Based Greenhouse**	**ESP32 + FreeRTOS + Blynk**	**Real-time multitasking, threshold irrigation, cloud monitoring, low cost**	**Future: AI-based predictive irrigation**

IV. Methodology

The proposed RTOS-Based Smart Greenhouse Monitoring System integrates environmental sensing, real-time processing, automated irrigation control, and cloud-based monitoring.

A. System Design Overview

The system follows an IoT-based layered architecture consisting of an environmental sensing layer, edge processing layer, communication layer, cloud monitoring layer, and user interface layer. Environmental parameters such as soil moisture, temperature, and humidity are continuously monitored. Based on real-time sensor readings, irrigation decisions are automatically executed.

B. Hardware Architecture

The hardware components in the system include an **ESP32 Microcontroller** as the central processing unit for multitasking and Wi-Fi communication, a **Soil Moisture Sensor** that measures volumetric water content, a **DHT22 Sensor** for ambient temperature and humidity, a **Relay Module** controlled by the ESP32 to switch the water pump, a **Water Pump** for irrigation, and a **Wi-Fi Router** to provide cloud connectivity.

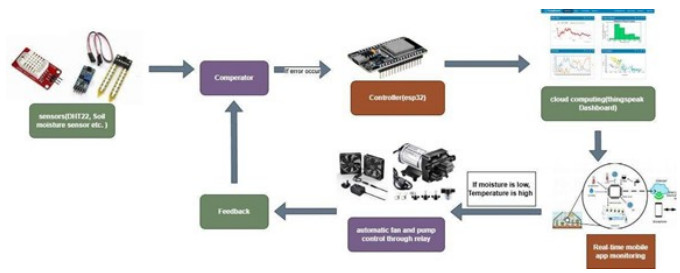

Fig. 1. Hardware Diagram

C. Software Architecture

The system software is developed using the Arduino IDE with FreeRTOS support on the ESP32. The control logic follows a threshold-based model: if the soil moisture falls below 30%, the pump is activated; otherwise, it remains off. Temperature and humidity data are also logged for environmental analysis.

D. RTOS Task Management

Under FreeRTOS, the system tasks are as follows: a Sensor Task that periodically reads the soil moisture, temperature, and humidity; a Control Task that compares the moisture reading against the threshold; a Pump Task that activates or deactivates the relay; a Communication Task that sends sensor data to the Blynk Cloud; and a Logging Task for serial debugging. Critical tasks (sensing and control) are given higher priority than communication tasks. FreeRTOS message queues are used to transfer sensor data to the control task, and semaphores synchronize pump activation, preventing resource conflicts.

E. System Flowchart

F. Data Communication and Cloud Integration

The ESP32 connects to Wi-Fi and transmits sensor data to the Blynk IoT Cloud every 2 seconds. The mobile application provides live sensor visualization, displays pump status, and

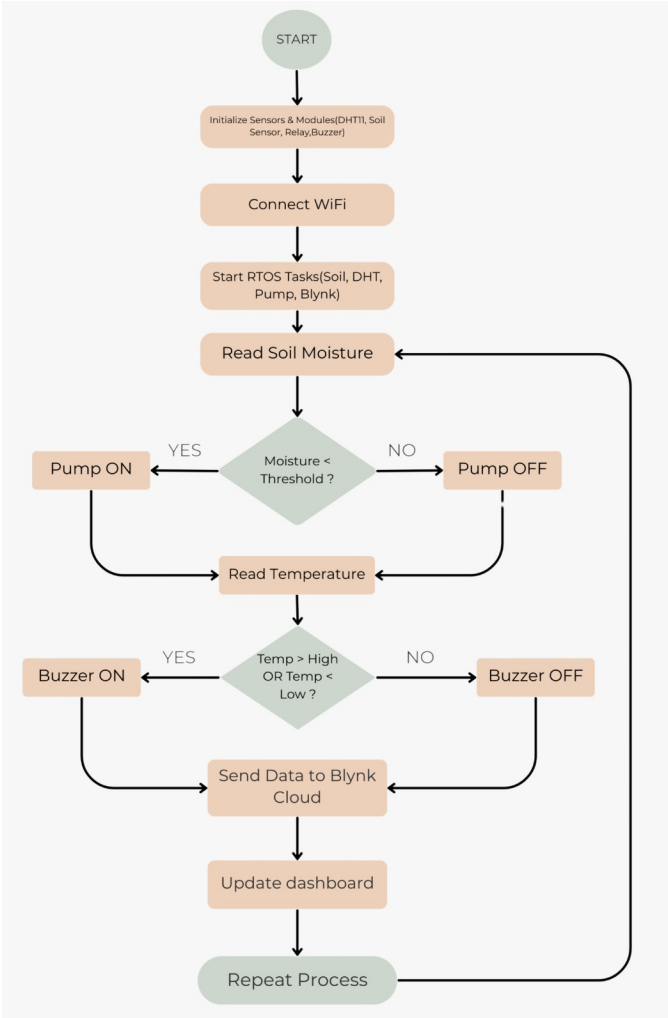

Fig. 2. Flowchart

offers manual pump control. It also sends a notification if the soil is dry. This setup enables remote monitoring, alerts, and user interaction with the greenhouse system.

G. Sampling and Timing Strategy

Sensors are sampled at 2-second intervals using FreeRTOS software timers instead of blocking delays. This improves responsiveness and minimizes idle CPU time. The 2-second sampling interval ensures fast detection of soil dryness while reducing unnecessary processing and power consumption.

H. Performance Evaluation Strategy

The system performance was evaluated under three conditions: (i) dry soil, where automatic irrigation is triggered; (ii) normal moisture, where only monitoring occurs; and (iii) network failure, where local automation continues without cloud connectivity. Performance metrics include response time, water usage efficiency, system reliability, and latency.

I. Power Management Consideration

Power consumption is optimized through efficient RTOS scheduling and minimizing idle CPU time. Tasks execute only when required, and blocking delays are replaced by timers to prevent CPU wastage. Sensors operate at a low duty cycle, and the pump is activated only when necessary. Future enhancements may include a solar-powered battery backup to support off-grid operation.

V. Mathematical Modeling

The control strategy and performance metrics are defined mathematically as follows. The pump activation model is:

$$P(t) = \begin{cases} 1, & \text{if } M(t) < M_{\text{threshold}}, \\ 0, & \text{otherwise}, \end{cases}$$

where $M(t)$ is the soil moisture percentage at time t and $M_{\text{threshold}}$ is the predefined threshold (30%). This ensures the pump turns on only when moisture is below the safe limit, conserving water.

Water usage efficiency compared to manual irrigation is calculated as:

$$\eta = \frac{W_{\text{manual}} - W_{\text{automatic}}}{W_{\text{manual}}} \times 100\%,$$

where W_{manual} is the total water used under manual control and $W_{\text{automatic}}$ is the water used by the automated system. Experimental results show approximately 25% water savings.

The sampling frequency f is given by

$$f = \frac{1}{T},$$

where T is the sampling interval (2 seconds, hence $f = 0.5$ Hz). This balances responsiveness and energy efficiency.

The total end-to-end latency is:

$$T_{\text{total}} = T_{\text{sensor}} + T_{\text{processing}} + T_{\text{transmission}},$$

where T_{sensor} is the sensor read delay, $T_{\text{processing}}$ is the RTOS decision time, and $T_{\text{transmission}}$ is the Wi-Fi/cloud delay. In practice, T_{total} is on the order of 1 second, confirming real-time performance.

VI. Network Architecture

The Smart Greenhouse system uses a layered IoT architecture for modularity and reliability. The *Sensing Layer* consists of the soil moisture sensor and DHT22 sensor. The *Edge Layer* is the ESP32 running FreeRTOS, which performs data acquisition and control. The *Communication Layer* uses Wi-Fi (TCP/IP) to send telemetry. The *Cloud Layer* is the Blynk platform for data storage, visualization, and alerts. The *Application Layer* is the mobile app interface for monitoring and control.

Each cloud update packet includes a timestamp, soil moisture (%), temperature, humidity, and pump status, sent every 2 seconds. This enables near real-time remote monitoring. If internet connectivity is lost, local threshold-based control continues normally; data synchronization resumes automatically once the connection is reestablished.

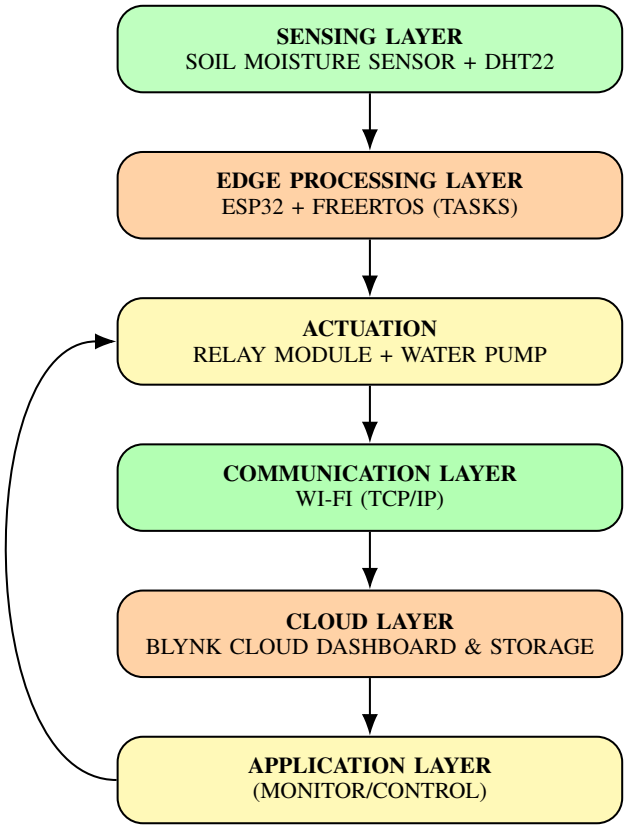

Fig. 3. Layered IoT architecture of the proposed Smart Greenhouse Monitoring System.

VII. Power Management

Power efficiency is critical for continuous operation. The instantaneous power is:

$$P_{\text{total}} = P_{\text{ESP32}} + P_{\text{sensors}} + P_{\text{relay}} + P_{\text{pump}},$$

where P_{ESP32}, P_{sensors}, P_{relay}, and P_{pump} are the powers of each component. The pump draws about 800 mA at peak, which is the largest load. During monitoring, power draw is much lower.

To minimize energy use, tasks execute only when needed, and timers replace idle delays to reduce CPU waste. Sensors sample at low frequency, and the pump activates only when required. These strategies keep average power consumption low:

$$P_{\text{avg}} = \frac{1}{T}\int_0^T P(t)\,dt.$$

Because the pump is on for only short intervals, P_{avg} is much lower than the peak. Future enhancements could include a solar panel and battery to enable off-grid power and further energy savings, as well as the ESP32 deep-sleep mode during extended idle periods.

VIII. System Implementation

The hardware prototype uses an ESP32 microcontroller, a resistive soil moisture sensor, a DHT22 sensor, a relay module, and a small water pump. The soil sensor output is connected to the ESP32 ADC (e.g., GPIO34), the DHT22 data pin to GPIO4, and the relay control to GPIO26. A 5V supply powers the ESP32 and peripherals, with proper isolation for the relay.

The firmware is implemented in the Arduino IDE with FreeRTOS. Initialization sets up GPIOs, ADC, Wi-Fi, and Blynk authentication. FreeRTOS tasks are created as described above. Queues transfer sensor values to the control task, and semaphores manage pump actuation.

The ESP32 establishes a Wi-Fi connection and authenticates with the Blynk cloud. Sensor readings are pushed to virtual pins. The mobile dashboard shows real-time soil moisture percentage, temperature, humidity, and pump status, along with a manual override button.

The system workflow is summarized in Algorithm 1. First, the ESP32 boots and connects to Wi-Fi/Blynk. Then it continuously: reads sensors, sends data to the cloud, compares moisture to threshold, and switches the pump ON or OFF accordingly, repeating every 2 seconds under FreeRTOS scheduling.

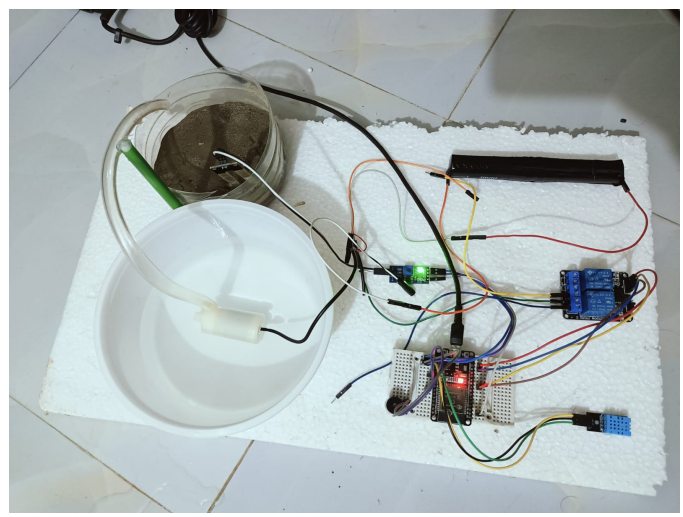

Fig. 4. Hardware Connection

The prototype was tested under various conditions. The pump reliably turned on when the moisture fell below 30% and off when moisture was restored. Cloud connectivity remained stable during extended operation. During a network outage, local control continued without interruption, and cloud synchronization resumed after reconnection, demonstrating robust hybrid operation.

IX. Experimental Results and Analysis

Tests show that the system responds within about 2 seconds when soil moisture falls below the threshold. Table II summarizes an example sequence, confirming timely pump activation and deactivation.

Water consumption was compared between manual irrigation and the automated system (Table **??**). Manual irrigation used 10.0 L, while the automated system used 7.5 L. This

Algorithm 1 RTOS-Based Smart Greenhouse Automation

```
Initialize ESP32, GPIO, ADC
Connect to Wi-Fi and authenticate with Blynk
Create FreeRTOS tasks (Sensor, Control, Pump, Communication)
while true do
    Read soil moisture, temperature, humidity
    Send data to Blynk
    if soil moisture < threshold (30%) then
        Activate relay (pump ON)
    else
        Deactivate relay (pump OFF)
    end if
    Delay until next 2-second interval
end while
```

TABLE II
SOIL MOISTURE RESPONSE OBSERVATION

Time (s)	Moisture (%)	Pump Status
0	45	OFF
4	38	OFF
8	29	ON
12	34	OFF
16	41	OFF

yields a water-saving efficiency of 25%. Latency measurements (sensor 100 ms, processing 50 ms, Wi-Fi 500 ms) yield an average end-to-end delay of about 1 second, confirming near real-time performance. During continuous operation, temporary network disconnections did not disrupt local irrigation control, and sensor monitoring continued uninterrupted. Cloud synchronization resumed automatically after reconnection.

The results indicate fast response to soil dryness, significant water savings, stable monitoring, and reliable RTOS-based multitasking.

X. COST ANALYSIS

Table III lists estimated component costs. The total is about $30, making the system affordable compared to commercial alternatives.

TABLE III
ESTIMATED COMPONENT COST

Component	**Qty**	**Cost (BDT)**
ESP32 Microcontroller	1	400
Soil Moisture Sensor	1	75
DHT22 Sensor	1	120
Relay Module	1	245
Water Pump	1	100
Power Supply	1	80
Misc. Components	-	20
Total		**1040**

The economic advantages include low initial investment, water savings (20–30%), reduced labor, and a scalable multi-plant design. These benefits lead to a short return on investment, making the system attractive for rural farmers.

XI. FUTURE WORK

Future enhancements include integrating AI-based predictive irrigation models to optimize watering schedules from historical data. A solar-powered battery backup would enable fully off-grid operation. Adding sensors for soil pH, electrical conductivity, and light intensity can provide richer environmental feedback. A GSM/GPRS module could offer a backup communication channel in areas without Wi-Fi. The mobile app will also be enhanced with advanced analytics and data logging features.

REFERENCES

[1] L. Zhang, Y. Chen, and M. Wang, "Design of an IoT-Based Smart Agriculture Monitoring System," *Int. J. Agricultural Technol.*, 2021.

[2] R. Kumar and P. Singh, "Automated Greenhouse Environment Control Using Embedded Systems," *Int. J. Smart Farming Technol.*, 2022.

[3] M. Ahmed, S. Rahman, and T. Islam, "Real-Time Environmental Monitoring Using FreeRTOS-Based Embedded Systems," *J. Embedded Syst.*, 2023.

[4] M. Danita, B. Mathew, N. Shereen, N. Sharon, and J. Paul, "IoT Based Automated Greenhouse Monitoring System," in *Proc. 2nd Int. Conf. on Intelligent Computing and Control Systems*, Madurai, India, 2018.

[5] R. Kirag and M. Tenruh, "Design and Implementation of a Cloud-Integrated, IoT-Enabled ESP32 Monitoring System," *Int. J. Comput. Appl.*, vol. 187, no. 11, 2025.

[6] B. T. de Abreu, F. M. P. de Castro, and M. G. Leizer, "Intelligent Plant Monitoring System for Greenhouse Deployment," *Revista Eng. Interesse Soc.*, vol. 5, no. 3, 2023.

[7] F. Abou-Mehdi-Hassani et al., "Systematic Literature Review of Smart Greenhouse Monitoring," *SN Comput. Sci.*, vol. 6, art. 95, 2025.

[8] F. R. Saputri et al., "Design and development of an irrigation monitoring and control system based on Blynk IoT and ThingSpeak," *PLoS One*, vol. 20, no. 4, e0321250, 2025.

[9] F. R. Shahroury, H. J. Abdellatif, and E. Al-Shaham, "Design and Deployment of an IoT-Enabled Wireless Sensor Network for Agriculture," in *Proc. Bridging Digital Innovation for Bus. Transf. (ICTIM)*, 2025, pp. 55–62.

[10] N. N. Thilakarathne et al., "IoT-enabled smart agriculture: Current status, latest advancements, challenges and countermeasures," *Heliyon*, vol. 11, e42136, 2025.

[11] K. Bouarroudj, F. Baba, and A. Touil, "IoT-based monitoring and control for optimized plant growth in smart greenhouses," *Internet of Things*, vol. 33, 101710, 2025.

[12] S. Banluesapy et al., "AI-Augmented Smart Irrigation System Using IoT and Solar Power for Sustainable Water and Energy Management," *Energy Eng.*, vol. 122, no. 10, 2025.

[13] V. K. Quy et al., "IoT-Enabled Smart Agriculture: Architecture, Applications, and Challenges," *Appl. Sci.*, vol. 12, no. 7, art. 3396, 2022.

[14] C. Talukdar and N. Singh, "FreeRTOS in IoT: A Comparative Study for Smart Agriculture," *Int. J. Real-Time Embedded Syst.*, vol. 14, no. 1, 2024.

[15] B. Yadav and A. Gupta, "Wireless Sensor Networks for Soil Moisture Monitoring in Precision Agriculture," *J. Smart Agric.*, vol. 7, no. 2, 2023.